

HTTP - a Hybrid Transformer for Trajectory Prediction

Arico Amaury
BruFace student

Colot Emmeran
BruFace student

Lungu Loric
BruFace student

Abstract—TODO

- I. INTRODUCTION
- II. STATE OF THE ART
- III. THE DATASET
- IV. OUR MODEL

The HTTP model presented in this paper uses both LiDAR points and pictures taken from the front camera in order to predict the trajectory of nearby vehicles. The inputs are refined using either the PointPillar model [2] for the LiDAR, or the ResNet-18 model [1] to process the pictures.

The extracted features are then fused inside a Transformer that produces a trajectory forecast over a horizon of 6 seconds.

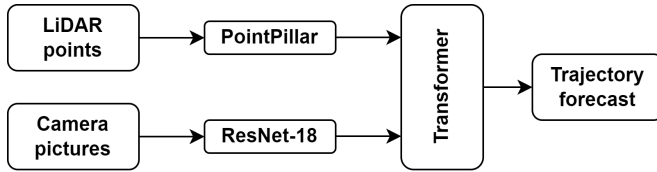


Fig. 1: General structure of the HTTP model.

Figure 1 showcases a block structure of the model and its different components are detailed hereafter.

A. PointPillar

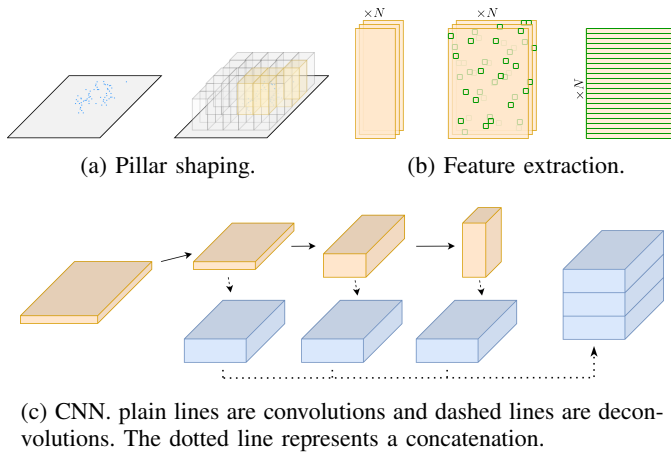


Fig. 2: PointPillar model overview.

The feature extraction of the PointPillar model is split into three successive stages, all shown in fig. 2.

The LiDAR points are first stacked in pillars based on their coordinates. A pillar contains at most 32 points and empty pillars are removed to reduce the memory footprint.

The second stage extracts features from the voxels by adding to each voxel its relative position to the pillar center and the pillar centroid. The number of features is then increased by passing through a two layers network, consisting of a linear operation followed by a batchnorm and a ReLU. Only the maximal feature over the voxels is then kept, leading to 64 features per pillar.

The last stage is a CNN, performing convolutions and deconvolutions on the pillars to extract data dependencies at different resolutions. Note that the pillars from the last stage are first placed on a grid matching the original position of the pillars.

The dimensions of the map on which pillars are placed were determined in order to contain all the voxels in the dataset.

B. Resnet-18

Rather than projecting the raw camera data directly into the target feature dimensions at the encoder, the input passes through feature extraction network introduced in the paper "Deep Residual Learning for Image Recognition" (He et al., 2015), known as **Resnet**. Historically, in 2015, image treatment was lead by the assumption that increasing network depth improves in a steady way the feature extraction capacity of the network. However, this idea hits severe optimization bottlenecks. First, the vanishing/exploding gradient during the backpropagation and second, a degradation of the training error once reaching a floor value. The latter degradation reveals a fundamental limitation of such deep network stacking linear transformation and non-linear activation function (e.g. ReLU) which is expressed, in the litterature, by the difficultly of performing an identity mapping for deep layers, or saying otherwise the ability to perform an identity transformation. To give an example, if a 30 layers Convolutional Neural Networks (CNNs) training leads to a lower error than a 50 layers counterpart, it implies that the 20 additional layers are, in the first place, unable to converge towards the same model accuracy, but furthermore, are distorting the output features by the inability of the NN layers to operate an identity transformation. To overcome these limiting performance constraints, **Resnet** introduces a direct path between the embedded input vector and the output of the feedforward performing a elementwise summation - Figure 3. This path, also known

as shortcut connection, make trivial the build of a identity mapping (no weighth paramater through the direct connection) while optimizing the features of the **input residual** - $\mathcal{F}(x)$.

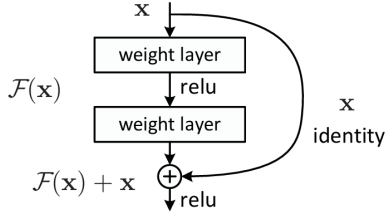


Fig. 3: Resnet block : residual learning.

The HTTP model incorporates a Resnet-18 architecture for image processing. As indicated by the naming, the network contains a total of **18** layers with learnable parameters. The image is first resized, normalized and centered around zero prior being propagated through the Resnet backbone. The output features are mapped via a linear layer and dimensioned to match the target size required for the embedding vectors at the encoder. Figure 4 showcases the network details.

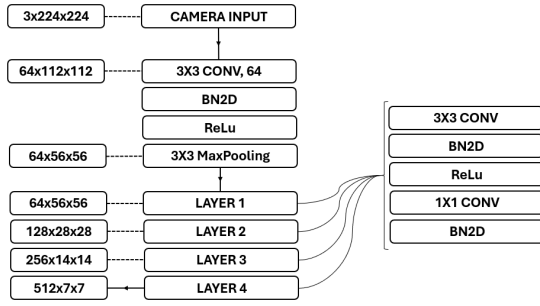


Fig. 4: Resnet18 network architecture.

It is worth noting that while the Resnet-18 has been built from scratch in a dedicated class for learning propurse, the model utilizes the standardized Resnet library enabling the integration of pre-trained weights reducing the training convergence time.

C. Long Short Term Memory

The last significant data still remaining at our disposal is the past trajectory history of the vehicle whose future path our model attempts to predict. This coordinate sequences carries critical information about the kinetic and momentum state of the vehicle. Nevertheless, these measurements are time-dependent and a standard feedforward linear NN is poorly suited to extract features spanning accross distinct timesteps. To efficiently extract these dynamics, our model introduces a Recurrent Neuronal Network block (**RNN**), and more specifically a Long Short Term Memory (**LSTM**) block - [3]. Unlike to a standard RNN relying to a single memory state (known as the hidden state, h_i), the LSTM decouples memory representation into a long term memory cell (C_i) and a short term memory state (h_i).

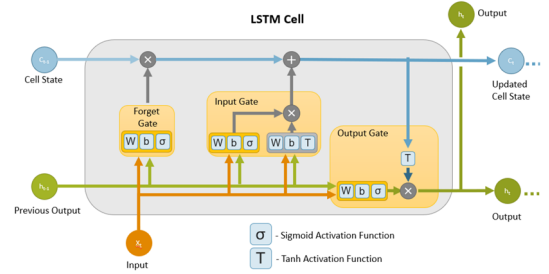


Fig. 5: LSTM network architecture.

The long term memory cell is linked with the short term memory state and the current input through **three gates**, namely the forget gate, the input gate and the candidate gate. These gates regulates how the input and the short term memory modulate the long term memory cell, C_t . It is achieved either by attenuating features by element-wise multiplication with the forgetting output, f_t or by adding new features through the combination of candidate gate output, $\tilde{c}_t$ and input gate output, m_t .

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (2)$$

$$m_t = \sigma(W_m \cdot [h_{t-1}, x_t] + b_m) \quad (3)$$

$$C_t = C_{t-1} \odot f_t + \tilde{c}_t \odot m_t \quad (4)$$

W_h , W_x , W_b , b_f , b_c and b_m denote respectively the weights matrix and the bias vector associated to the gates, h_{t-1} the previous hidden state, x_t the current input.

The primary advantage of such architecture lies in its stability against the vanishing/exploding gradient descent which is caused, for a standard RNN, by the recursive update of the hidden state at each time step leading making this long-term temporal dependency sensitive to such gradient problematic.

For the standard RNN, The loss gradient of the hidden state is computed by repeated weight matrix multiplication, W_h , leading to a vanishing or exploding gardient depending on the largest matrix eigenvalue value.

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad (5)$$

$$\frac{\partial \varepsilon}{\partial h_k} = \frac{\partial \varepsilon}{\partial h_t} \cdot \prod_{j=k+1}^t \text{term}(\tanh(\alpha)) \cdot W_h^T \quad (6)$$

W_h and W_x denote the weights matrix associated to the hidden state and the input, b the bias, ε the error loss and α the term inside the function $\tanh$, Eq. 5.

In case of LSTM block, the gradient computation is significantly different and depends on the dynamic activation vector (f_j) programmed by the network. This vector varies in function of the current input and the previous hidden state injected in the block.

$$\frac{\partial \varepsilon}{\partial \mathcal{C}_k} = \frac{\partial \varepsilon}{\partial \mathcal{C}_t} \cdot \prod_{j=k+1}^t \frac{\partial \mathcal{C}_t}{\partial \mathcal{C}_{t-1}} \quad (7)$$

$$\frac{\partial \mathcal{C}_t}{\partial \mathcal{C}_{t-1}} \approx f_t \quad (8)$$

Regarding our model implementation, upstream of the LSTM block, the original past positional history input (4 frames x 2 coordinates) is first processed by a 2-layer Linear network expanding the 2 coordinates features into 64 features then 256 features space. The hidden state vector and the long term memory Cell vector are also initialized as a zero vector. Then, the LSTM block is instantiated by the *CustomLSTMCell* class building a feedforward Linear network mapping the concatenated input features dimension and the hidden state vector dimension into combined four channel hidden state tensor. The stacking of the weight matrices of each gate into an unified block helps saving time computation and optimizing the GPU usage. By sequential application of the gate output equations, the LSTM block returns a post-processed hidden state vector output composed of 256 features.

D. Transformer

E. Inference and Accuracy Metrics

V. RESULTS

VI. CONCLUSIONS

REFERENCES

- [1] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- [2] Alex H. Lang et al. “PointPillars: Fast Encoders for Object Detection from Point Clouds”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 12697–12705. DOI: [10.1109/CVPR.2019.01298](https://doi.org/10.1109/CVPR.2019.01298).
- [3] Christian Bakke Vennerød, Adrian Kjærran, and Erling Stray Bugge. “Long Short-term Memory RNN”. In: *CoRR* abs/2105.06756 (2021). arXiv: [2105.06756](https://arxiv.org/abs/2105.06756). URL: <https://arxiv.org/abs/2105.06756>.